

```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoCreate;  
4 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoCreateConUsuario;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoDto;  
6 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoEdit;  
7 import com.ejemploSpring.Ejercicio.  
   entities.Pedido;  
8 import com.ejemploSpring.Ejercicio.  
   service.PedidoService;  
9 import lombok.RequiredArgsConstructor  
   ;  
10 import org.springframework.web.bind.  
   annotation.*;  
11  
12 import java.util.List;  
13  
14 @RestController  
15 @RequestMapping("/pedidos")  
16 @RequiredArgsConstructor  
17 public class PedidoController {  
18  
19     private final PedidoService  
       pedidoService;
```

```
20
21     // Crear pedido con detalles
22     @PostMapping
23     public PedidoDto create(@
    RequestBody PedidoCreate pedidoCreate
    ) {
24         return PedidoDto.toDto(
    pedidoService.save(pedidoCreate));
25     }
26
27     // Obtener pedido por ID
28     @GetMapping("/{id}")
29     public PedidoDto findById(@
    PathVariable Long id) {
30         return PedidoDto.toDto(
    pedidoService.findById(id));
31     }
32
33     // Obtener todos los pedidos
34     @GetMapping
35     public List<PedidoDto> findAll
    () {
36         return pedidoService.findAll
    ().stream()
37             .map(PedidoDto::toDto
    )
38             .toList();
39     }
40
```

```
41      // Editar pedido (fecha, estado,
      forma de pago)
42      @PutMapping("/{id}")
43      public PedidoDto update(@
      PathVariable Long id, @RequestBody
      PedidoEdit pedidoEdit) {
44          return PedidoDto.toDto(
      pedidoService.update(id, pedidoEdit
      ));
45      }
46
47      // Soft delete
48      @DeleteMapping("/{id}")
49      public void delete(@PathVariable
      Long id) {
50          pedidoService.deleteById(id
      ); // baja lógica
51      }
52
53      @PostMapping("/asignar")
54      public PedidoDto
      crearPedidoConUsuario(
55          @RequestBody
      PedidoCreateConUsuario dto
56      ) {
57          Pedido pedido = pedidoService
      .save(dto.pedido());
58          pedidoService.asignarPedido(
      dto.usuarioId(), pedido);
```

```
59         return PedidoDto.toDto(pedido
    );
60     }
61
62 }
```

```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioCreate;  
4 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioDto;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioEdit;  
6 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioNombreDto;  
7 import com.ejemploSpring.Ejercicio.  
   service.UsuarioService;  
8 import jakarta.validation.Valid;  
9 import lombok.RequiredArgsConstructor  
   ;  
10 import org.springframework.http.  
   HttpStatus;  
11 import org.springframework.web.bind.  
   annotation.*;  
12  
13 import java.util.List;  
14  
15 @RestController  
16 @RequestMapping("/usuarios")  
17 @RequiredArgsConstructor  
18 public class UsuarioController {  
19  
20     private final UsuarioService
```

```
20 usuarioService;
21
22     // Get All
23     @GetMapping("")
24     public List<UsuarioDto>
25     getUsuarios() {
26         return usuarioService.findAll
27         ();
28     }
29
30     // Get By ID
31     @GetMapping("/{id}")
32     public UsuarioDto getUsuario(@
33     PathVariable Long id) {
34         return usuarioService.
35         findById(id);
36     }
37
38     // GET solamente nombre y
39     apellido
40     @GetMapping("/{id}/nombre")
41     public UsuarioNombreDto
42     getNombreApellido(@PathVariable Long
43     id) {
44         return usuarioService.
45         getNombreApellido(id);
46     }
47
48     // Create
```

```
41     @PostMapping("")
42     @ResponseStatus(HttpStatus.
    CREATED)
43     public UsuarioDto createUsuario(@
    Valid @RequestBody UsuarioCreate
    usuarioCreate) {
44         return usuarioService.save(
    usuarioCreate);
45     }
46
47     // Update
48     @PutMapping("/{id}")
49     public UsuarioDto updateUsuario(
50         @PathVariable Long id,
51         @Valid @RequestBody
    UsuarioEdit usuarioEdit
52     ) {
53         return usuarioService.update(
    usuarioEdit, id);
54     }
55
56     // Delete
57     @DeleteMapping("/{id}")
58     @ResponseStatus(HttpStatus.
    NO_CONTENT)
59     public void deleteUsuario(@
    PathVariable Long id) {
60         usuarioService.deleteById(id
    );
```

61 }

62

63 }

64


```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3 import com.ejemploSpring.Ejercicio.  
   dtos.producto.ProductoCreate;  
4 import com.ejemploSpring.Ejercicio.  
   dtos.producto.ProductoDto;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.producto.ProductoEdit;  
6 import com.ejemploSpring.Ejercicio.  
   service.ProductoService;  
7 import jakarta.validation.Valid;  
8 import lombok.RequiredArgsConstructor  
   ;  
9 import org.springframework.http.  
   HttpStatus;  
10 import org.springframework.web.bind.  
   annotation.*;  
11  
12 import java.util.List;  
13  
14 @RestController  
15 @RequestMapping("/productos")  
16 @RequiredArgsConstructor  
17 public class ProductoController {  
18  
19     private final ProductoService  
       productoService;  
20
```

```
21      // Get All
22      @GetMapping("")
23      public List<ProductoDto>
    getProductos() {
24          return productoService.
    findAll();
25      }
26
27      @GetMapping("/eliminados")
28      public List<ProductoDto>
    getEliminados() {
29          return productoService.
    findAllEliminados();
30      }
31
32      // GetByID
33      @GetMapping("/{id}")
34      public ProductoDto getProducto(@
    PathVariable Long id) {
35          return productoService.
    findById(id);
36      }
37
38      // Create
39      @PostMapping("")
40      @ResponseStatus(HttpStatus.
    CREATED)
41      public ProductoDto createProducto
    (@Valid @RequestBody ProductoCreate
```

```
41 productoCreate) {
42     return productoService.save(
        productoCreate);
43 }
44
45 // Update
46 @PutMapping("/{id}")
47 public ProductoDto updateProducto
    (
48     @PathVariable Long id,
49     @Valid @RequestBody
        ProductoEdit productoEdit
50 ) {
51     return productoService.update
        (productoEdit, id);
52 }
53
54 // Delete
55 @DeleteMapping("/{id}")
56 @ResponseStatus(HttpStatus.
    NO_CONTENT)
57 public void deleteProducto(@
    PathVariable Long id) {
58     productoService.deleteById(id
    );
59 }
60 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  controller;  
2  
3  
4 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaCreate;  
5 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaDto;  
6 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaEdit;  
7 import com.ejemploSpring.Ejercicio.  
  service.CategoriaService;  
8 import lombok.RequiredArgsConstructor  
  ;  
9 import org.springframework.web.bind.  
  annotation.*;  
10  
11 import java.util.List;  
12  
13 @RestController  
14 @RequestMapping("/categorias")  
15 @RequiredArgsConstructor  
16 public class CategoriaController {  
17  
18     private final CategoriaService  
        categoriaService;  
19  
20     // Crear categoría  
21     @PostMapping
```

```
22     public CategoriaDto create(@
    RequestBody CategoriaCreate
    categoriaCreate) {
23         return categoriaService.save(
    categoriaCreate);
24     }
25
26     // Obtener categoría por ID (solo
    activas)
27     @GetMapping("/{id}")
28     public CategoriaDto findById(@
    PathVariable Long id) {
29         return categoriaService.
    findById(id);
30     }
31
32     // Obtener todas las categorías
    activas
33     @GetMapping
34     public List<CategoriaDto> findAll
    () {
35         return categoriaService.
    findAll();
36     }
37
38     // Editar categoría
39     @PutMapping("/{id}")
40     public CategoriaDto update(@
    PathVariable Long id,
```

```
41                                     @
    RequestBody CategoriaEdit
    categoriaEdit) {
42        return categoriaService.
    update(categoriaEdit, id);
43    }
44
45    // Soft delete
46    @DeleteMapping("/{id}")
47    public void delete(@PathVariable
    Long id) {
48        categoriaService.deleteById(
    id);
49    }
50
51 }
```